CUDA-Accelerated Ray Tracer with Multi-GPU Rendering

CS/EE 147 — Spring 2026 Final Project

Chris Antony Kaushik Vada

Last updated: May 28, 2026

Abstract

This is a living report for our CUDA ray tracer. The renderer loads a triangle mesh (an OBJ exported from Fusion 360) and shades it with diffuse, metal, and dielectric materials. We build a CPU baseline, then port it to the GPU and layer on a bounding volume hierarchy and dual-GPU rendering, benchmarking four configurations against each other. The document is updated phase by phase; each new configuration appends a row to the benchmark table in Section 4. As of this revision the CPU baseline (with mesh loading) and the naive CUDA port are complete, and the naive GPU port is roughly three orders of magnitude faster than the single-threaded CPU on the same scene.

1 Overview

The project compares four rendering configurations end to end:

1. **CPU single-threaded** — baseline, a port of Peter Shirley's *Ray Tracing in One Weekend* extended with triangle-mesh support.
2. **CUDA naive** — one GPU thread per pixel, brute-force ray/scene intersection.
3. **CUDA + BVH** — a bounding volume hierarchy for sub-linear ray/scene intersection.
4. **Dual-GPU** — framebuffer split across two NVIDIA A30s.

The end goal is a tool we can hand an arbitrary Fusion 360 export and get a rendered image, fast enough that mesh density is not a practical limit. Configurations 3 and 4 are what make that true; configurations 1 and 2 establish the baseline and the correctness reference.

2 Pipeline and implementation

2.1 Geometry and mesh loading

Fusion 360 exports meshes as triangle soup, so the core primitive is a single triangle intersected with the Möller–Trumbore algorithm. Each triangle stores its three vertices, an optional set of per-vertex normals, and a material index. When the OBJ provides vertex normals (`vn`) we interpolate them across the triangle for smooth shading; otherwise we fall back to the flat geometric normal.

The OBJ loader parses `v`, `vn`, and `f` records, handles the `v`, `v/vt`, `v//vn`, and `v/vt/vn` face formats including negative (relative) indices, and fan-triangulates any polygon face into $n - 2$

triangles. It also returns the mesh axis-aligned bounding box, which the camera uses to auto-frame the model: the look-at point is the box center and the camera is pulled back along a fixed three-quarter view by a distance that fits the bounding sphere in the vertical field of view. This means any export lands on screen regardless of its scale or origin.

2.2 CPU baseline

The baseline keeps the original header-only structure (`inc/`), using `double` precision, `std::shared_ptr` for scene objects, and virtual dispatch for the `hittable` and `material` hierarchies. It is single-threaded and serves as both configuration 1 and the correctness reference for the GPU path.

2.3 CUDA naive port

The GPU path is a separate type stack under `cuda/` rather than a recompile of the CPU headers, for three reasons:

- **No virtual dispatch.** Virtual functions and `shared_ptr` do not translate to device code. Materials become a single plain-old-data struct with a type tag (lambertian / metal / dielectric); each triangle stores an index into a flat material array and the scatter routine switches on the tag.
- **Single precision.** The A30's FP64 throughput is a fraction of its FP32, and path tracing does not need the extra precision, so the GPU types use `float`.
- **Iterative, not recursive.** The CPU `ray_color` recurses per bounce. GPU stacks are small, so the kernel folds attenuation forward in a bounded loop instead of recursing.

The scene (a flat triangle array plus the material array) is uploaded to device memory once. The render kernel launches one thread per pixel on an 8×8 block grid; each thread seeds a cuRAND state, accumulates `samples_per_pixel` jittered samples, and writes the averaged color to the framebuffer, which is copied back and written as a gamma-corrected PPM. Intersection in this configuration is a brute-force linear scan over every triangle — this is the cost the BVH in Phase 3 removes.

3 Benchmark methodology

Hardware. NVIDIA A30 (compute capability 8.0), CUDA 13.2 toolkit. CPU times are single-threaded.

Scene. A UV sphere of 4608 triangles with per-vertex normals (smooth shading), rendered at 800×450, 20 samples per pixel, maximum ray depth 10. The same OBJ and camera framing are used for both configurations.

What is timed. The GPU figure is kernel time (init plus render), measured with a host-side high-resolution clock around the launch and `cudaDeviceSynchronize`. The CPU figure is whole-process wall-clock time; mesh parsing for this scene is well under a tenth of a second, so the CPU number is effectively render time.

4 Results

Table 1 is the running benchmark. Rows for configurations 3 and 4 are filled in as those phases land.

Table 1: Render time on the 4608-triangle sphere (800×450, 20 spp, depth 10). Speedup is relative to the CPU baseline.

Configuration	Time (s)	Speedup (×)	Status
CPU single-threaded	585.23	1	**done**
CUDA naive	0.47	1245	**done**
CUDA + BVH	—	—	**pending**
Dual-GPU	—	—	**pending**

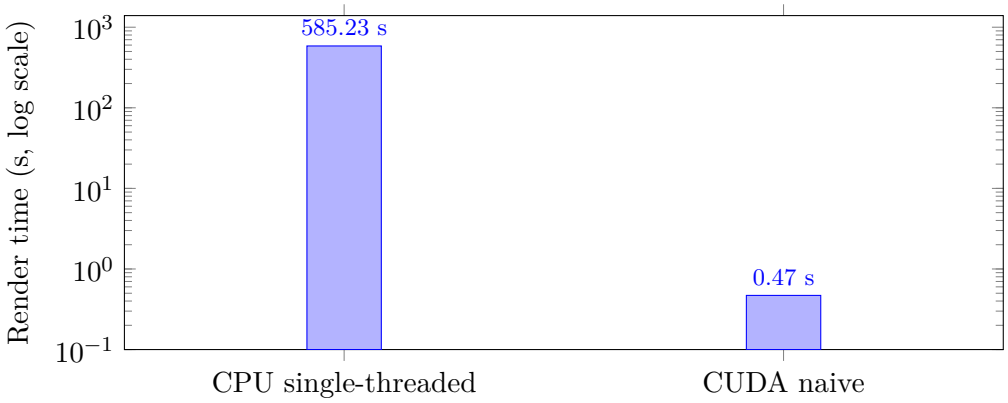

Figure 1: CPU baseline vs. naive CUDA on the benchmark scene. Note the log axis — the naive GPU port is about 1245× faster.

4.1 Correctness validation

The GPU output is validated against the CPU baseline on the same scene. Because the two paths use different random number generators (`std::rand` vs. cuRAND) and different floating-point precision, they do not produce bit-identical images; instead they produce the same image with independent Monte-Carlo noise. On the sphere scene the per-channel mean absolute difference is about 0.2 out of 255, with isolated single-pixel maxima around 34 — the signature of sampling noise, not a geometric or shading discrepancy. Figure 2 shows the two side by side; Figure 3 shows the flat-shaded path on a quad-faced cube.

5 Status and roadmap

Table 2: Phase status.

Phase	Deliverable	Status
1	CPU baseline + OBJ mesh loading	**done**
2	CUDA naive port (one thread/pixel)	**done**
3	BVH acceleration	**pending**
4	Dual-GPU rendering	**pending**
5	Benchmarks + final writeup	**pending**

The single most important remaining piece for the project's stated goal is the BVH (Phase 3).

(a) CPU baseline (b) CUDA naive

Figure 2: Smooth-shaded 4608-triangle sphere (interpolated vertex normals). CPU baseline (left) and naive CUDA (right) are visually identical; the only difference is independent sampling noise.

(a) CPU baseline (b) CUDA naive

Figure 3: Flat-shaded unit cube (quad faces, fan-triangulated, no vertex normals), exercising auto-framing and the flat-normal path.

Both the CPU baseline and the naive GPU port scale linearly with triangle count, so a full-detail Fusion part is impractical until ray/scene intersection drops to sub-linear cost. Dual-GPU (Phase 4) then splits the framebuffer across both A30s for a further constant-factor speedup.

6 Reproducing these numbers

```
# build (nvcc must be on PATH for the GPU target)
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j

# render the benchmark scene
./build/main      sphere.obj > cpu.ppm   # config 1
./build/main_cuda sphere.obj > gpu.ppm   # config 2
```

Both binaries print timing and triangle counts to `stderr` and the PPM image to `stdout`.